The journal’s data

Every file and table the journal keeps, who writes it, who reads it, and which of it is used.

Written from `journal/src/db/index.js`, `journal/src/journal/db.js`, `journal/src/trading/db.js`, `grading.js`, `checks.js`, `volumeBaseline.js`, `journal/server.js` and `journal/src/addons/accounts.js`. The table list was taken from a fresh database built by the real code: **30 tables** (31 with SQLite’s own `sqlite_sequence`).

Files

File	What	Notes
`data/journal/journal.db`	SQLite, WAL mode, foreign keys on	path: `JOURNAL_DB`, else `<JOURNAL_DATA_DIR>/journal.db`, else `<repo>/data/journal/journal.db`. Backed up nightly as `data/journal/journal.db.gz`
`data/journal/journal-accounts.json`	the Alpaca accounts with their keys	mode 0600, gitignored, **not** backed up (ACCOUNTS.md)
`data/keys.json`	the desk’s shared keys	read only as a fallback Alpaca key for bars
`public/desk.css`, `public/desk.js`, `tools.config.json`	the shared app bar and the registry	read from the repo on every request

Where the database came from. The journal used to be a separate install (`~/journal-app`) whose database was `tradedesk.db`, from an older all-in-one app. `deploy-tools.sh` stops the old process and **copies** that database (and the accounts file) once into `data/journal/` when no `journal.db` exists, leaving the old install in place as a backup. The code still creates that old app’s whole schema, so a fresh database has 30 tables: 13 are the journal’s own (described below, some never written), and 17 are leftovers of that old app.

The tables the journal uses

`journal_trades`: one row per trade

Column	Type	Meaning
`id`	text, key	file import and by hand: random uuid. Alpaca: `alpaca:<account id>:<first fill id>`
`date`	text	the **entry's** New York date, `YYYY-MM-DD`
`ticker`	text	upper case. File imports from TradingView keep the exchange prefix
`direction`	text	`Long` or `Short`
`setup_id`	text	the setup tag, set from the page
`shares`	integer	
`entry_price`, `exit_price`	real	exit blank while open
`entry_time`, `exit_time`	text	New York clock: `HH:MM` (imports) or `HH:MM:SS` (Alpaca). **No exit date**
`sl`, `tp`	real	stop and target. **Never set** by any import or fetch
`gross_pnl`, `commission`, `net_pnl`	real	net = gross − commission. Commission is 0 for Alpaca, DXtrade and the paper log
`pct_move`	real	% move; signed by direction except for file imports
`duration_ms`	integer	
`mae_pct`, `mfe_pct`, `r_multiple`, `capture_pct`, `exit_verdict`		written by the technical compute (ANALYSIS.md); R, capture and verdict need a stop
`source`	text	`import`, `alpaca` or `manual`
`account`	text	the label typed on import, or the Alpaca account's **name**
`notes`	text	API only
`status`	text	`open` or `closed`
`technical_computed`	integer	1 once the compute ran; reset to 0 when exit, stop or target is edited

Column	Type	Meaning
`created_at`	integer	ms since 1970

`journal_technical_snapshots`: the entry, as computed

One row per trade (`trade_id` is the key). Columns: `price`, `vwap`, `ema9`, `ema13`, `ema20`, `ema50`, `sma5`, `sma9`, `sma13`, `sma20`, `bb_upper`, `bb_mid`, `bb_lower`, `atr` (never filled), `rvol` (never filled), `pm_high`, `pm_low` (never filled), `price_rel_vwap`, `price_rel_ema9/13/20/50` (`above` / `below` / `at`), `ema_stack_bullish` (1/0), `bb_zone` (`upper` / `lower`), `ma5day`, `ma5day_slope`, `prev_day_high`, `prev_day_low` (never filled), `vwap_2day`, `vwap_week`, `vwap_month`, `week_high`, `week_low`, `month_high`, `month_low` (from the one day, look-ahead), `open`, `high`, `low`, `volume`, `prev_close` (the entry bar's and the bar before it), `sec_hot` (never filled), `computed_at`.

`journal/server.js` adds an `id` column at start-up that is always blank. An old route asked for it, and its absence had broken the condition step after tagging a setup.

`journal_context_snapshots`: the market at the trade

`trade_id` (key), `regime`, `regime_label`, `sec_bias`, `broad_resolved`, `short_term`, `mid_term`, `long_term`, `sec_score`, `sector`, `industry`, `scanner_score`, `scanner_confidence`, `captured_at`. Written from `r1_frozen` (never filled) or by two routes nothing calls. **Always empty in practice.**

`journal_conditions`: a setup template's conditions, judged

`id`, `trade_id`, `condition_key`, `label`, `section`, `value`, `aligned` (1, 0 or blank), `mandatory`. Rewritten for a trade each time it is evaluated. **Nothing reads it.**

`journal_setup_templates`

`id`, `setup_id` (unique), `conditions` (JSON list), `min_grade_trades` (20), `min_condition_trades` (10), `updated_at`. Written only by `PUT /api/journal/templates/:setupId`. The two minimums are never read.

`trading_setups`: the setup list

`id`, `name`, `description`, `indicator`, `entry_type`, `window_start`, `window_end`, `enabled`, `config` (JSON), `override_kill_switch`, `auto_pause_c_streak`. The page's setup dropdown shows the **enabled** rows.

Nothing in the journal writes this table. A fresh database has **no setups**, so no trade can be tagged, nothing reaches the grading copy, and the Setups tab stays empty. Only a database copied from the old app has setups (D-J14).

`trade_cards` and `trade_card_checks`: the grading copy

`trade_cards`: `id` (= the trade's id), `date`, `ticker`, `setup_id`, `direction`, `account`, `position_id`, `order_id` (always blank here), `entry_price`, `exit_price`, `shares`, `stop_distance`, `r_multiple`, `net_pnl`, `exit_reason`, `fired_at`, `closed_at`, `context` (JSON, with `source: "journal"`), `grade`, `grade_details` (blank here).

`trade_card_checks`: `id`, `card_id`, `kind`, `check_key`, `label`, `section`, `value`, `aligned`, `check_version_id`, `variation` (`symmetric` / `long` / `short`). Deleting a card deletes its checks.

Written by the grading bridge (ANALYSIS.md). Read by nothing.

`check_library` and `setup_check_assignments`

`check_library`: `id`, `check_key` (unique), `label`, `category` (`default` / `additional`), `section`, `description`, `condition`, `condition_long`, `condition_short` (JSON), `direction`, `enabled`, `version_id`, `created_at`, `updated_at`. **77 seeded rows**, topped up at every start.

`setup_check_assignments`: `setup_id`, `check_id`, `created_at`. Which additional checks a setup carries. No route writes it.

`journal_daily_notes`, `journal_fee_profiles`

`journal_daily_notes`: `date` (key), `note`, `updated_at`. `journal_fee_profiles`: `id`, `account` (unique), `rules` (JSON), `mode` (`replace`), `updated_at`. Both written only through their routes. No screen uses them, and **fee rules are never applied** to any trade (D-J15).

`settings`

46 keys. The journal reads only `alpacaApiKey`, `alpacaApiSecret` and `alpacaMarketFeed` (that last one is not seeded), and only as fallbacks for the bar fetch. The rest are listed below as leftovers.

Leftovers from the old app (created, never used)

Tables	From
`shortlist`, `r0_checkpoint`, `r1_frozen`, `r2_market_snapshots`, `r3a`, `r3b`, `r4a_train`, `r4b_train`, `scheduler_jobs`, `analysis_model`	the screener tool's schema, as it was in `tradedesk.db` (`journal/src/db/index.js`)
`trading_sessions`, `trading_signals`, `trading_orders`, `trading_signal_log`, `trading_positions`, `trading_grading_events`	the old live trading engine, removed from the desk (`journal/src/trading/db.js`)
`volume_baseline`	the RVOL baseline; created, never filled

Settings	From
21 seeded by `journal/src/db/index.js`: hot-sector and sector thresholds, shortlist rules, scorer entry time, regime sample size, Finnhub key, Alpaca key, secret and account URL, analysis options, AI key and model	the screener tool
25 seeded by `journal/src/trading/db.js`: `trading_*` (risk %, share and dollar caps, equity, data source, execution mode, live confirmation, market gate) and `grading_*` (7 thresholds used only by the uncalled grading functions)	the old live trading engine

`r1_frozen` matters because two journal features read it: the context snapshot and the pre-market levels. The screener tools freeze R1 in their own databases, never in the journal's, so both are always empty.

What survives what

Event	Effect
restart of the journal process	nothing lost: everything is in `journal.db` and the accounts file. The volume baseline and history cache live in memory and are empty anyway
`git pull` / deploy	nothing touched: `data/` is gitignored
nightly backup	`journal.db` is copied (gzipped); the accounts file is not, by design
restore	`journal.db` comes back; the Alpaca accounts must be added again on the page

What a rebuild should hold

One trade table with an **entry and exit timestamp** (not one date and two clock times), a stop and target on every trade, the fills that made it, fees as fees, and a stable external id. The journal's own setups, as a table the page can edit. Snapshots computed as of the entry. None of the leftover tables.